What does that look like in a session? Let's pick up our Pry session, already in progress:

```
[1] pry(main)> self
=> main
[2] pry(main)> cd register
[3] pry(#<CashRegister>):1> self
=> #<CashRegister:0x000000010277ecc0
 @items=[{:item=>"pen", :quantity=>3, :price_in_cents=>499}],
 @tax_rate=0.05>
[4] pry(#<CashRegister>):1> cd @items
[5] pry(#<Array>):2> size
=> 1
[6] pry(#<Array>):2> cd first
[7] pry(#<Hash>):3> keys
=> [:item, :quantity, :price_in_cents]
[8] pry(#<Hash>):3> cd ..
[9] pry(#<Array>):2>
```

The session starts out with the scope pointed at main. On line 2, we change to register, a local variable in scope. We can prove this on line 3 by checking the value of self. Note the number after the colon in the prompt, which tells us how many levels deep we are. On line 4 we change scope again, this time to @items, an instance variable that is part of register. Then on line 5, we show that size is sent to the item in scope, effectively @items.size, giving us 1. Then we cd again, this time to first, the first item of that array, and we show that keys is sent to that hash. Then on line 9, we use cd .. to pop back up a level to the array scope.

At any time you can use the command nesting to see a list of the current stack of contexts, and the jump-to n command to immediately go back up the stack to a particular point in that stack. Intervening contexts are exited.

Pry gives you some ways to explore the context that it is currently embedded in. The main way is with the ls method. The ls method gives you access to basically all of the ways Ruby lets you learn things about an item. You can see a list of the instance variables of an object, but also the methods available from that object.

If you type ls without an argument, it will give you that list on the current context, but if you give it an object argument, as in ls items, Pry will display the information for that object. There are a number of options for how to limit the fire hydrant of information, see https://github.com/pry/pry/wiki/State-navigation#Ls for a full list.

Pry also gives you a lot of ways to learn about the code you are looking at. You can type find-method with a method name, and Pry will search the Ruby libraries to show you what classes define methods that include that string in the name. In the following code we see that String and Symbol both define the method upcase:

```
[2] pry(main)> find-method upcase
String
String#upcase
String#upcase!
Symbol
Symbol#upcase
```

Pry's most distinctive feature is its ability to show you the source code for methods that you might be looking at. The show-source command takes a method or class name, and if that name is defined in the current context, shows you the source code: